Colección Postman

1. Archivo de colección Postman (.json)

Login exitoso

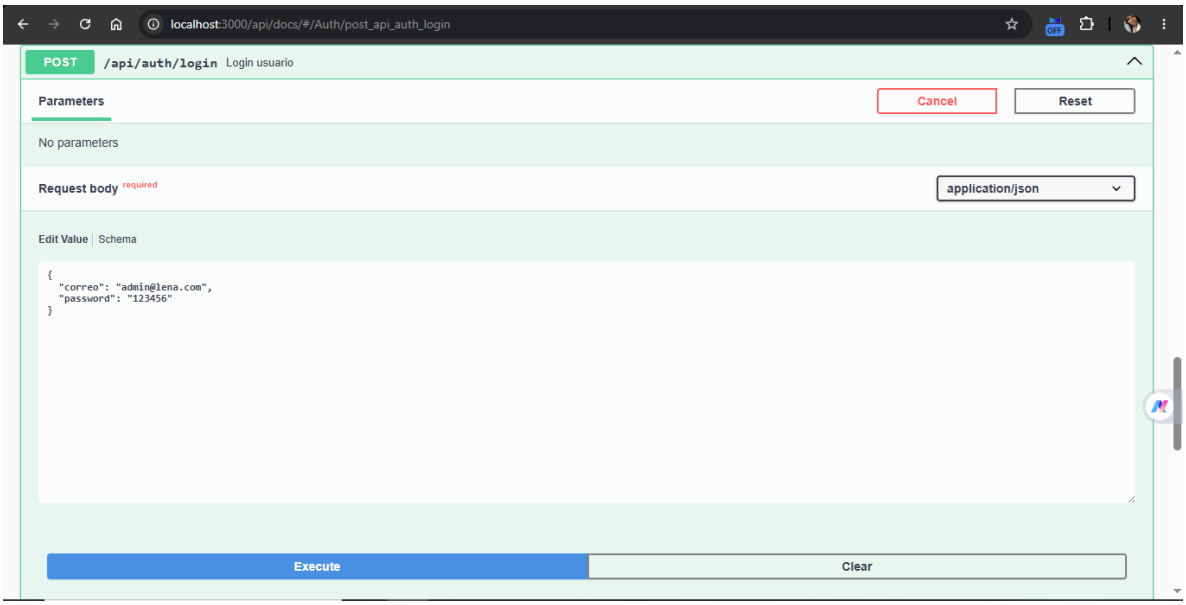
- Método: POST
- Endpoint:

  /api/auth/login

Se comprueba el funcionamiento del sistema de autenticación que utiliza JWT, logrando obtener un token válido para ingresar a los puntos finales protegidos.

Figura 1

Página principal de la API Leña Reserva

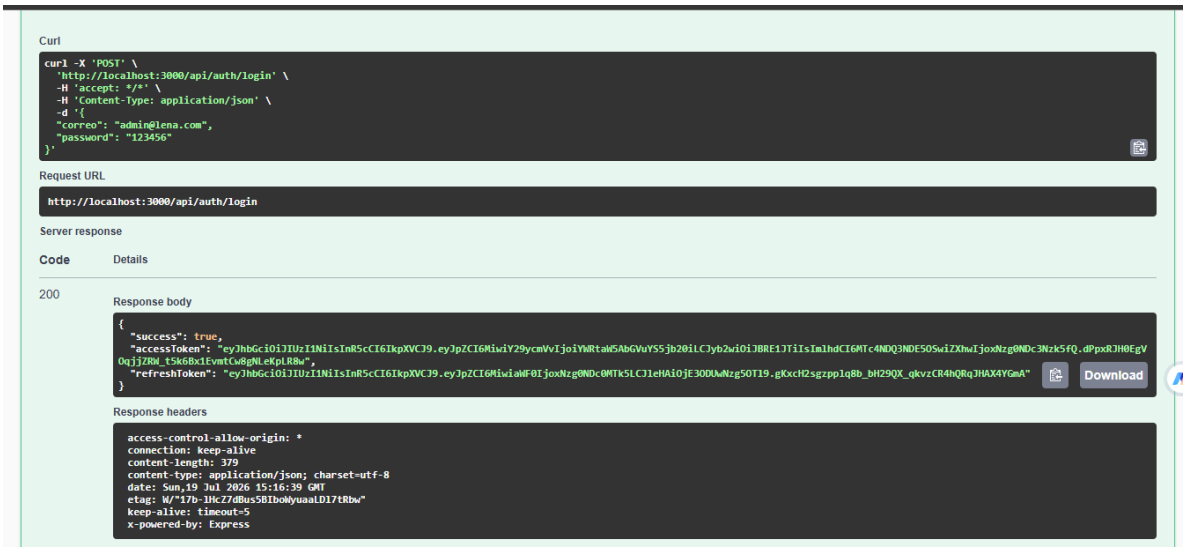


Nota: Vista inicial de la API que muestra información general sobre el proyecto y los servicios que ofrece (Vargas,2026).

La figura 2 que sigue ilustra la interfaz de Swagger, que se usa para documentar y poner a prueba los servicios REST de la API. Esta herramienta ayuda a verificar los endpoints y a observar su funcionamiento.

Figura 2

Documentación Swagger de la API



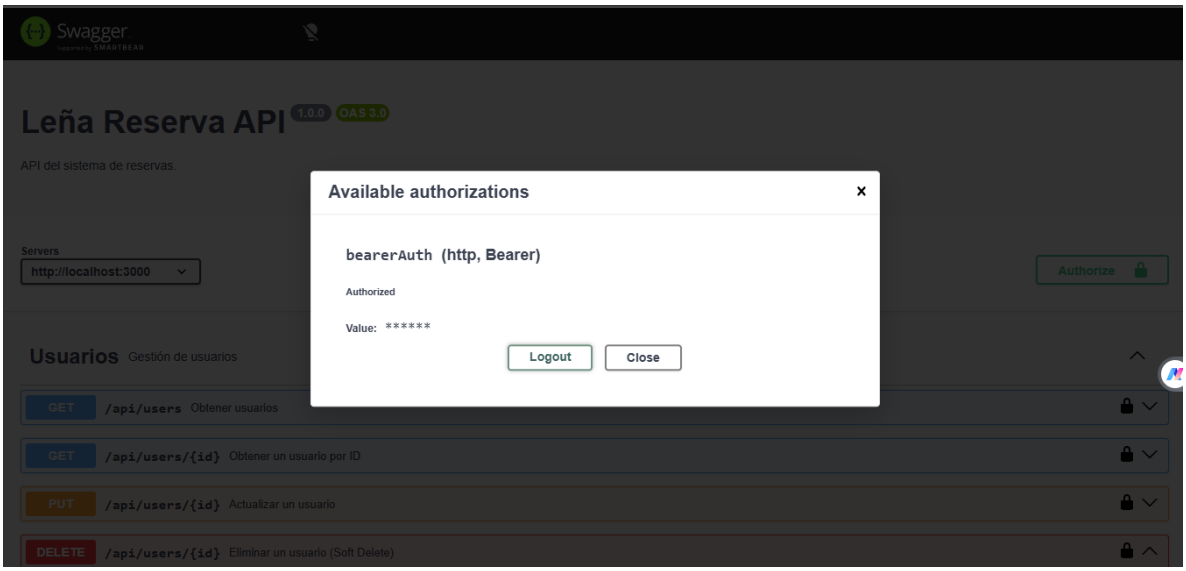
Nota: Interfaz de Swagger utilizada para la consulta y prueba de los puntos finales del sistema (Vargas,2026).

Respuesta con token JWT

El token creado durante el inicio de sesión se emplea para verificar las solicitudes seguras a través de un middleware de autenticación.

Figura 3

Inicio de sesión mediante JWT



Nota: Autenticación del usuario administrador para recibir el token de acceso (Vargas,2026).

GET Usuarios

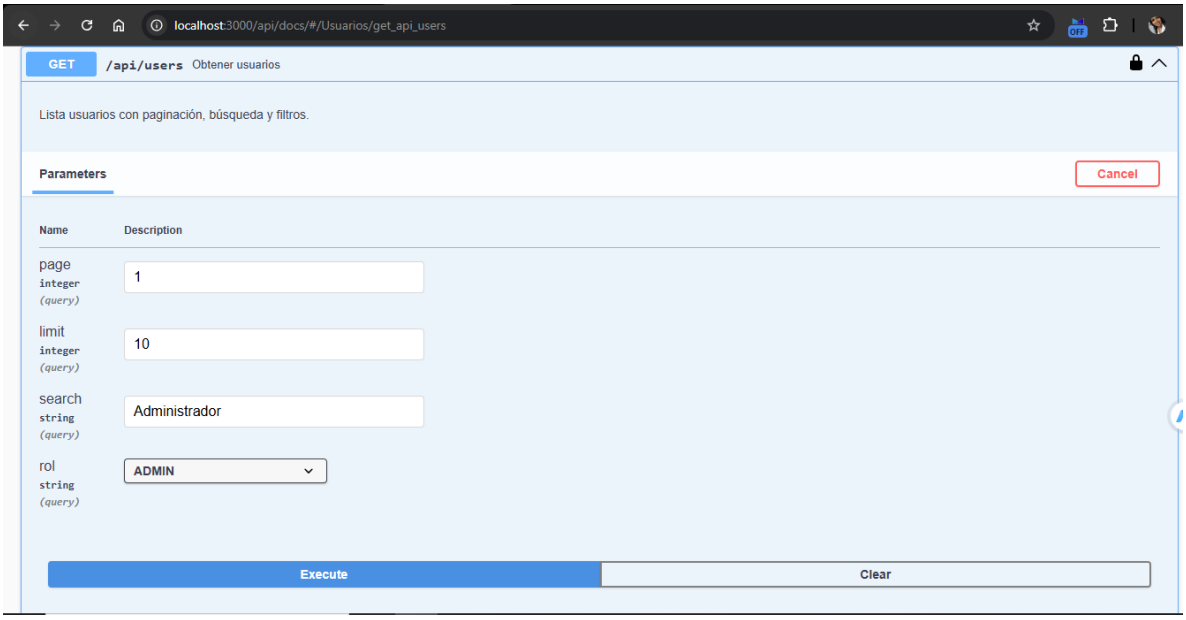
Se verifica la consulta de usuarios guardados en la base de datos utilizando Prisma ORM de manera paginada.

Endpoint:

GET /api/users

Figura 4

Listado de usuarios

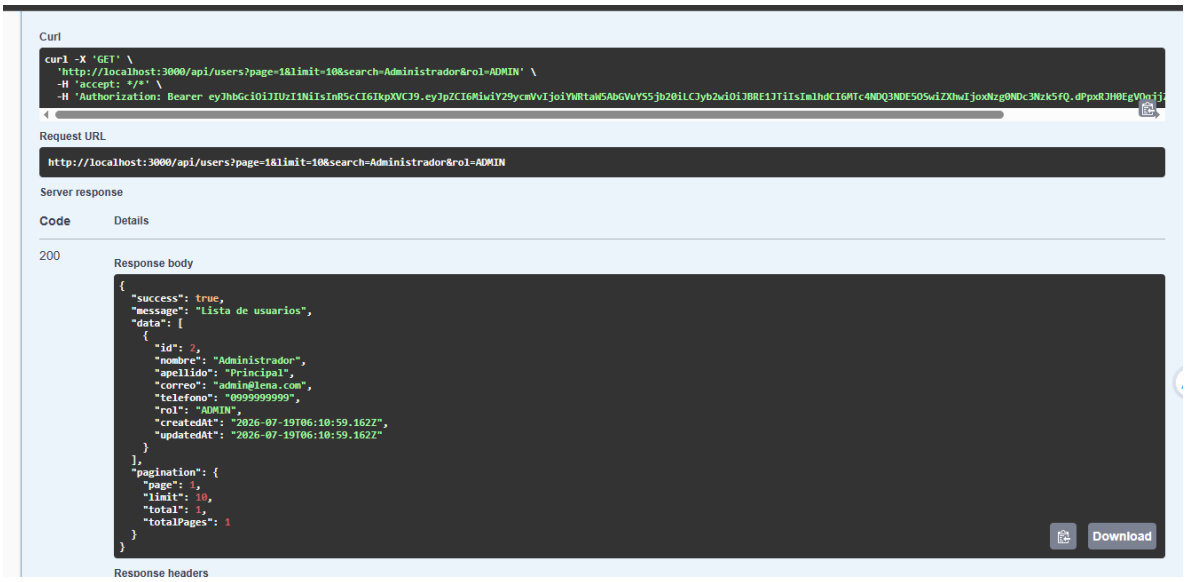


Nota: Consulta de los usuarios que ya están registrados, recibiendo una respuesta afirmativa del servido (Vargas,2026).

La figura 5 que se presenta a continuación está relacionada con el proceso de inscribir a un nuevo usuario en el sistema. Se asegura que los datos ingresados se guarden correctamente en la base de datos.

Figura 5

Registro de un usuario



Nota: Creación de un nuevo perfil de usuario a través del endpoint apropiado (Vargas,2026).

CRUD Mesas

Se demuestra la operación total del sistema CRUD de mesas, lo que posibilita el ingreso, la consulta, la modificación y la eliminación de datos.

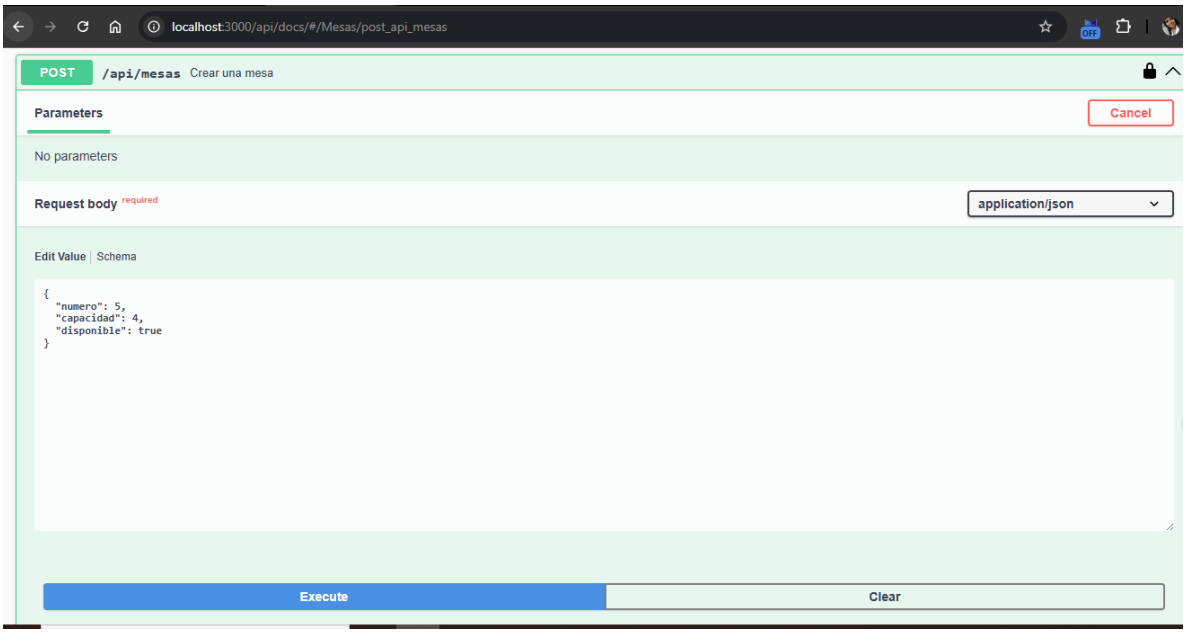
Crear mesa

Endpoint:

POST /api/mesas

Figura 6

Registro de una mesa



Nota: Adición de una nueva mesa dentro del sistema de reservas (Vargas,2026).

En la figura 7 siguiente refleja la confirmación de la inscripción de una mesa en el sistema. Esta acción permite añadir nuevos espacios que estarán disponibles para reservas futuras.

Figura 7

Creación de mesa exitosa

Nota: Confirmación del correcto registro de la mesa en la base de datos (Vargas,2026).

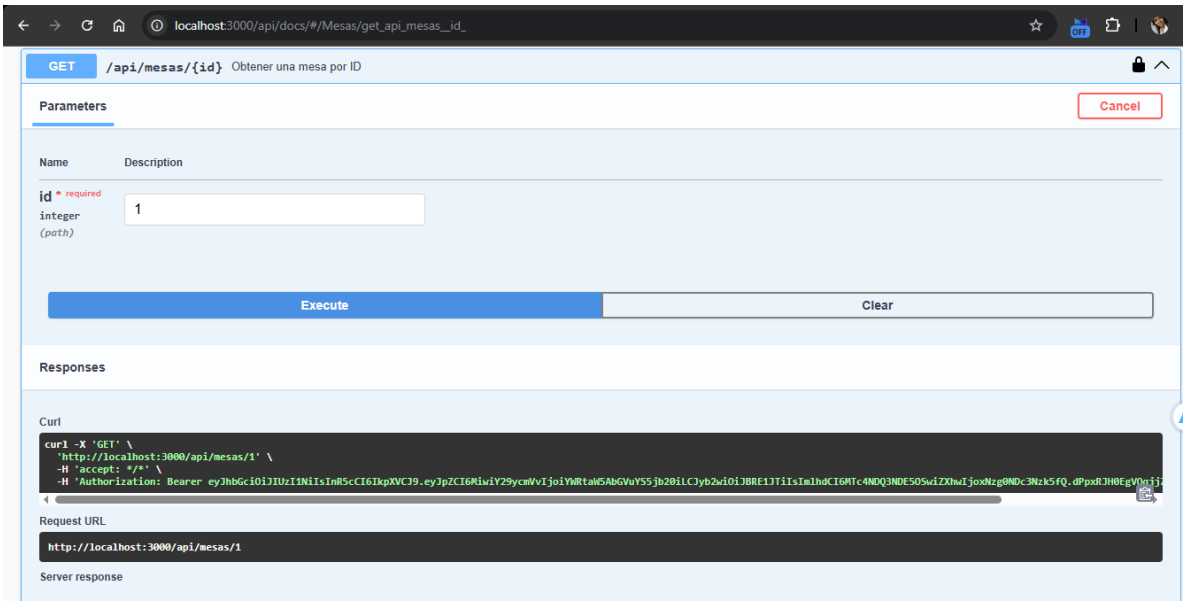
Consultar mesa

Endpoint:

GET /api/mesas/1

Figura 8

Listado de mesas

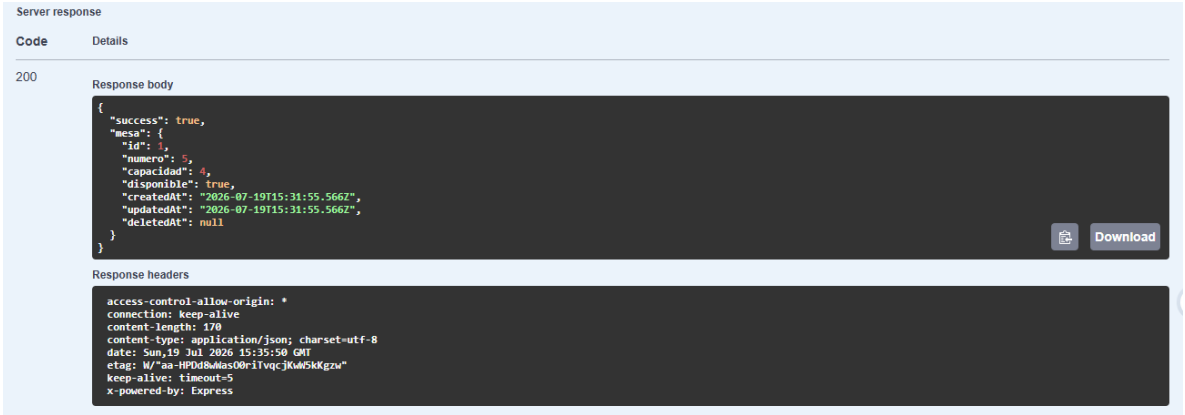


Nota: Consulta de todas las mesas que están registradas en la base de datos (Vargas,2026).

La evidencia a continuación muestra la consulta de una mesa específica utilizando su identificador, en la figura 9. Esto permite verificar que el sistema recupera adecuadamente la información guardada.

Figura 9

Consulta de mesa por ID



Nota: Exposición de la información de una mesa específica (Vargas,2026).

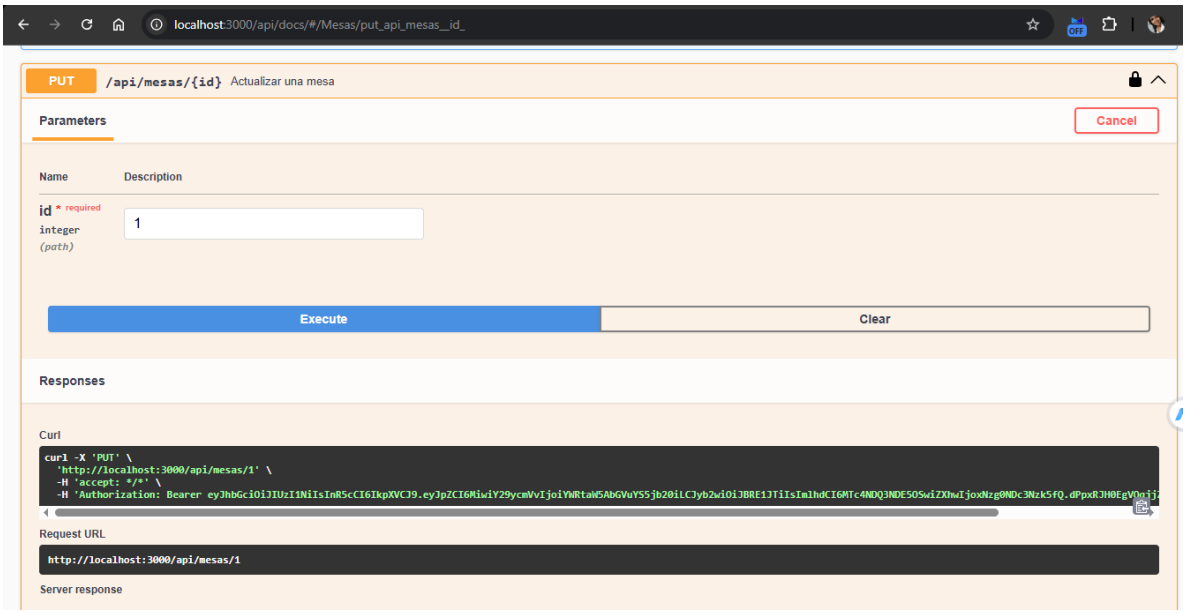
Actualizar mesa

Endpoint:

PUT /api/mesas/1

Figura 10

Actualización de una mesa

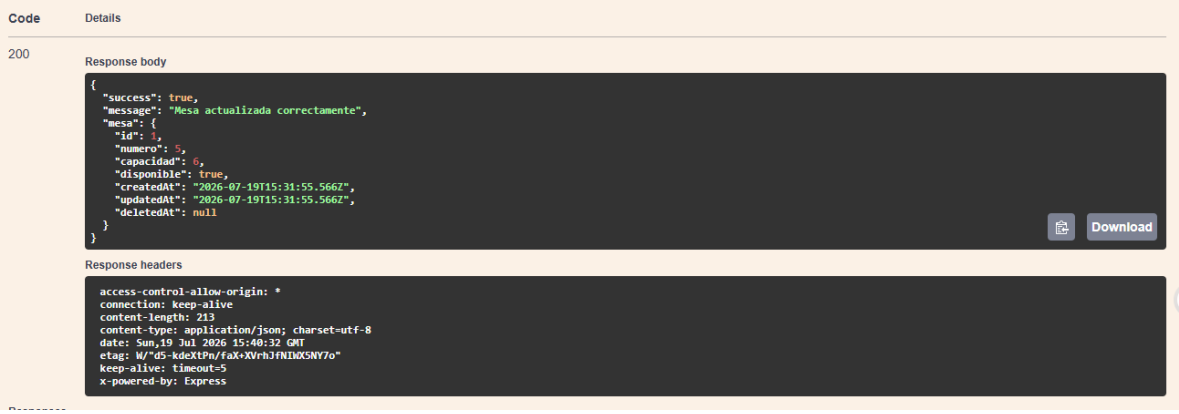


Nota: Cambios en la información relacionada con una mesa específica (Vargas,2026).

La figura 11 que sigue el resultado que se obtiene tras actualizar la información de una mesa. Esto sirve para confirmar que los cambios efectuados fueron correctamente registrados.

Figura 11

Actualización de mesa exitosa



Nota: Verificación de que la mesa seleccionada se ha actualizado correctamente (Vargas,2026).

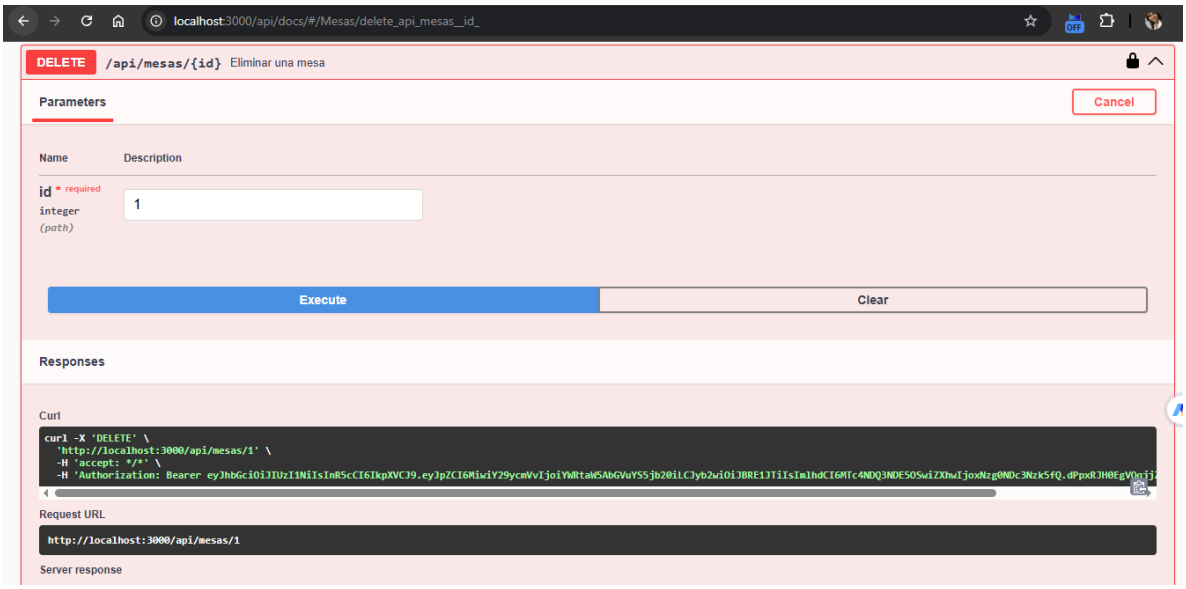
Eliminar mesa

Endpoint:

DELETE /api/mesas/5

Figura 12

Eliminación de una mesa



Nota: Eliminación lógica de una mesa que ya estaba registrada en el sistema (Vargas,2026).

La evidencia en la figura 13 que sigue se relaciona con la eliminación lógica de una mesa del sistema. Este proceso mantiene el registro en la base de datos sin que aparezca como disponible.

Figura 13

Eliminación de mesa exitosa

Nota: Confirmación de que la mesa seleccionada ha sido eliminada lógicamente (Vargas,2026).

CRUD Reservas

Se verifica el desempeño del sistema de reservas, abarcando las conexiones entre usuarios, mesas y estados de reserva.

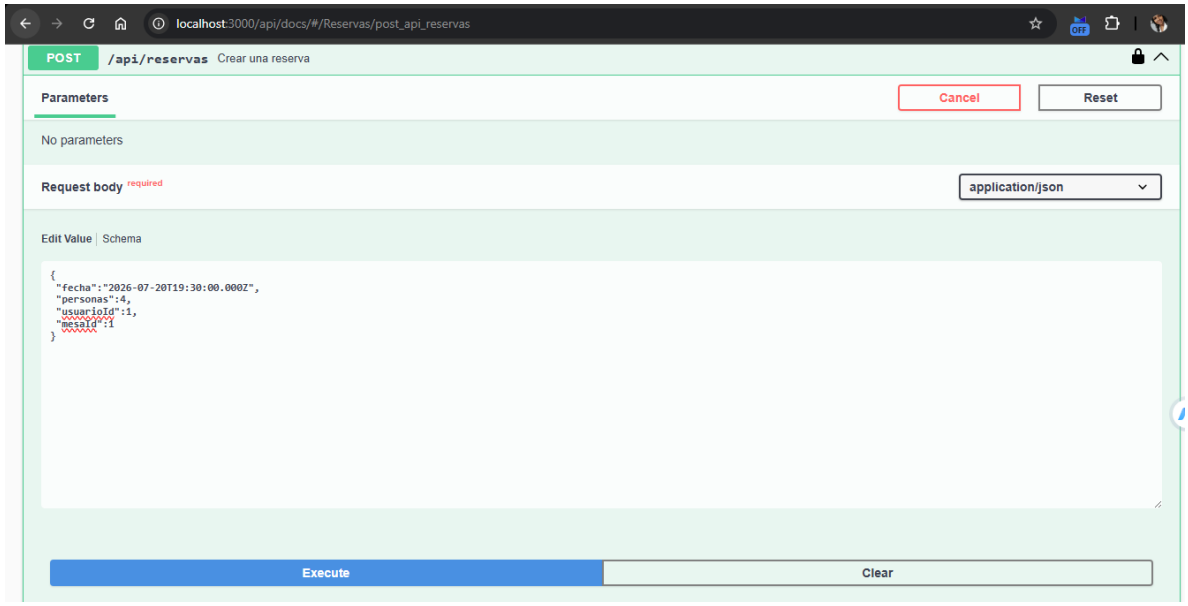
Crear reserva

Endpoint:

POST /api/reservas

Figura 14

Creación de una reserva

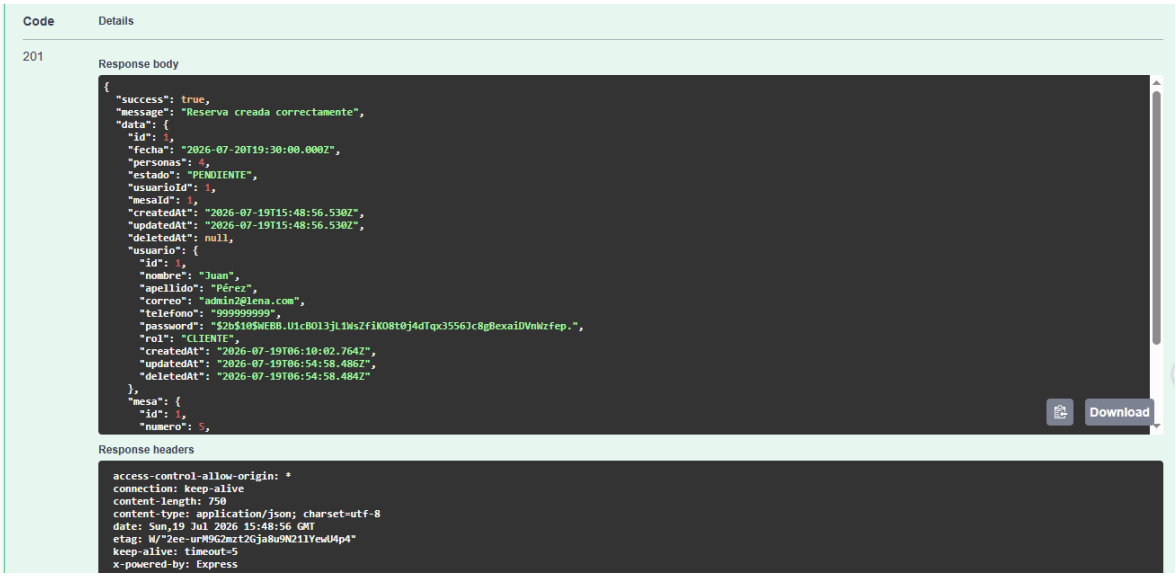


Nota: Creación de una nueva reserva vinculada a un usuario y a una mesa que está disponible (Vargas,2026).

La figura 15 siguiente muestra el resultado de crear una reserva en el sistema. Se valida la correcta conexión entre el usuario, la mesa y la fecha escogida.

Figura 15

Reserva creada correctamente



Nota: Confirmación del éxito en el almacenamiento de la reserva (Vargas,2026).

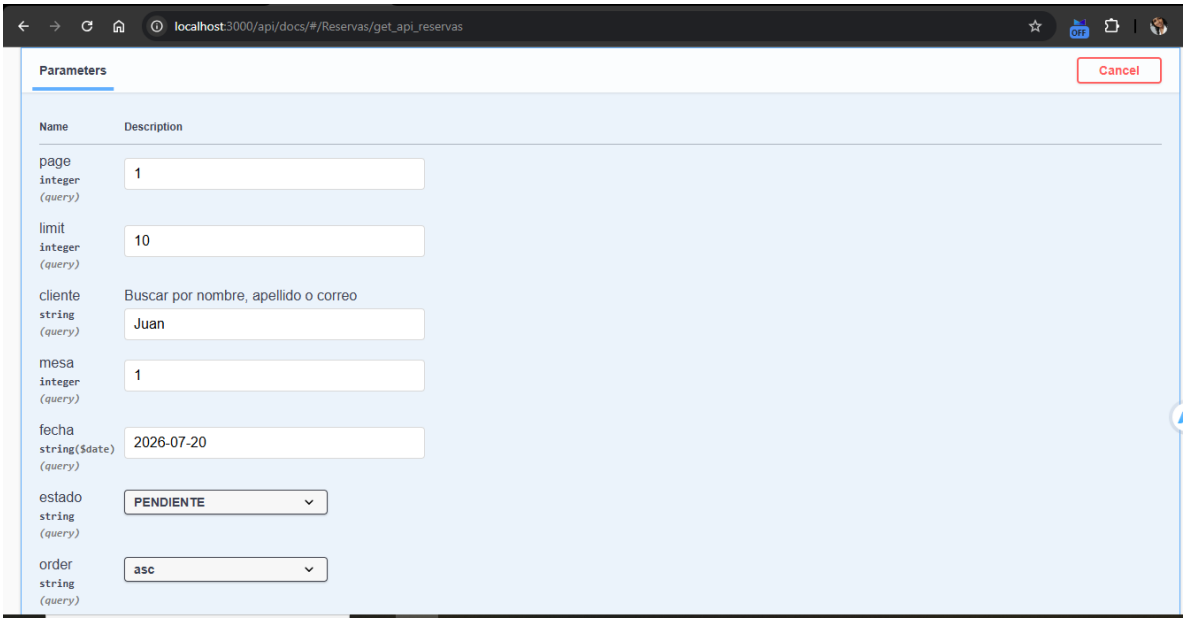
Listar reservas

Endpoint:

GET /api/reservas?page=1&limit=10

Figura 16

Listado de reservas

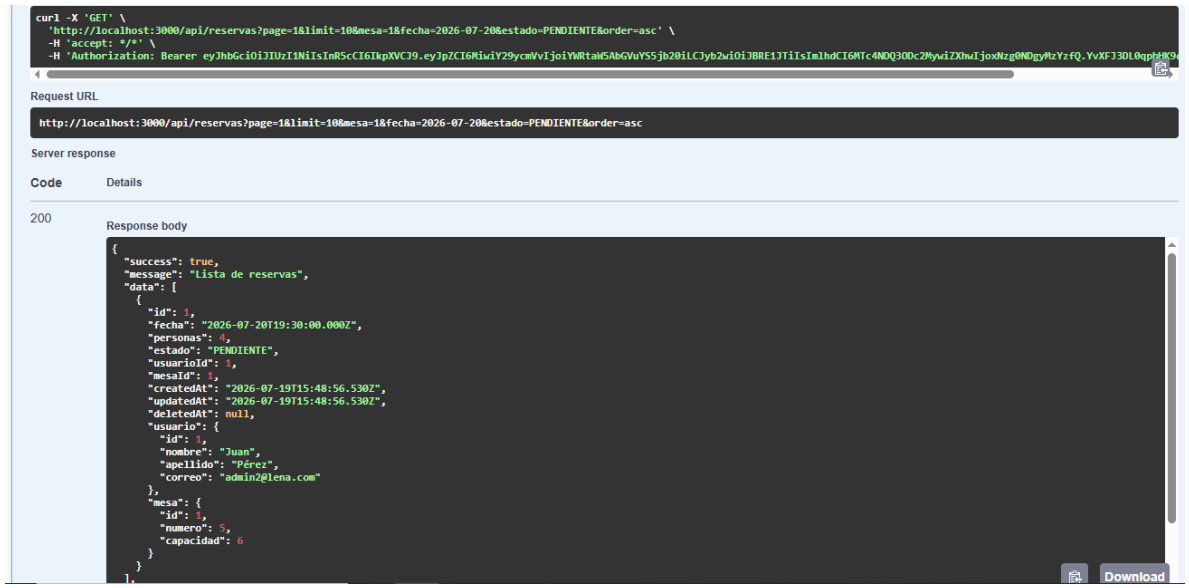


Nota: Consulta de las reservas registradas junto con su información asociada (Vargas,2026).

La evidencia a continuación presenta la consulta de una reserva que se había registrado anteriormente. Esto asegura que la información vinculada se recupera correctamente desde la base de datos, en la figura 17.

Figura 17

Consulta de reserva



Nota: Visualización de los detalles de una reserva en particular (Vargas,2026).

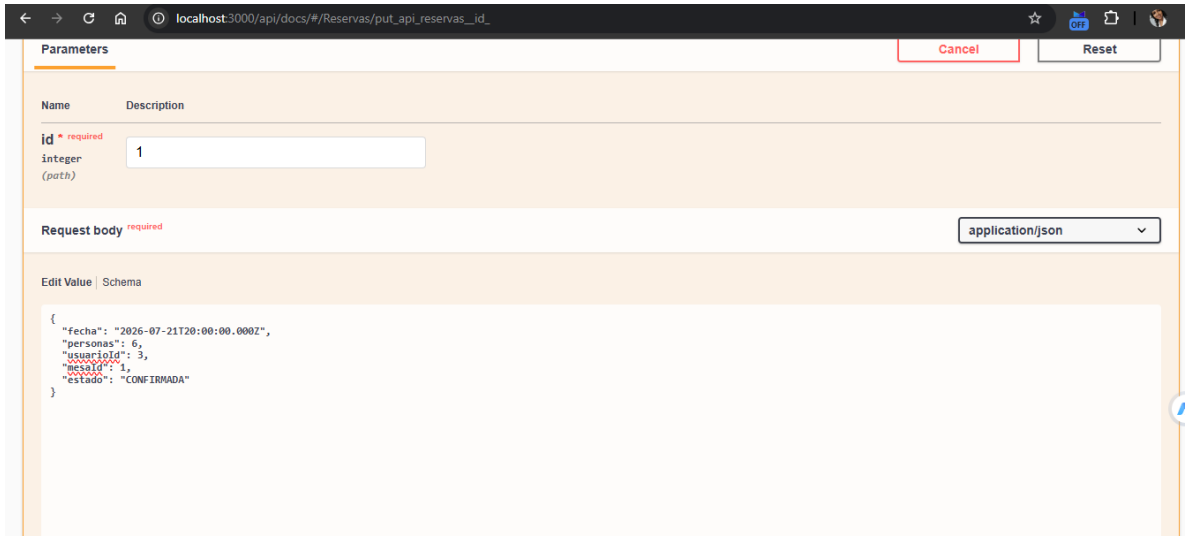
Actualizar reserva

Endpoint:

PUT /api/reservas/1

Figura 18

Actualización de una reserva



Nota: Cambios en los datos asociados a una reserva que ya existe (Vargas,2026).

La figura 19 que sigue se refiere al resultado de la actualización de una reserva ya existente. Se comprueba que las alteraciones realizadas se guardan correctamente en el sistema.

Figura 19

Reserva actualizada correctamente

Nota: Confirmación de que la información de la reserva fue actualizada (Vargas,2026).

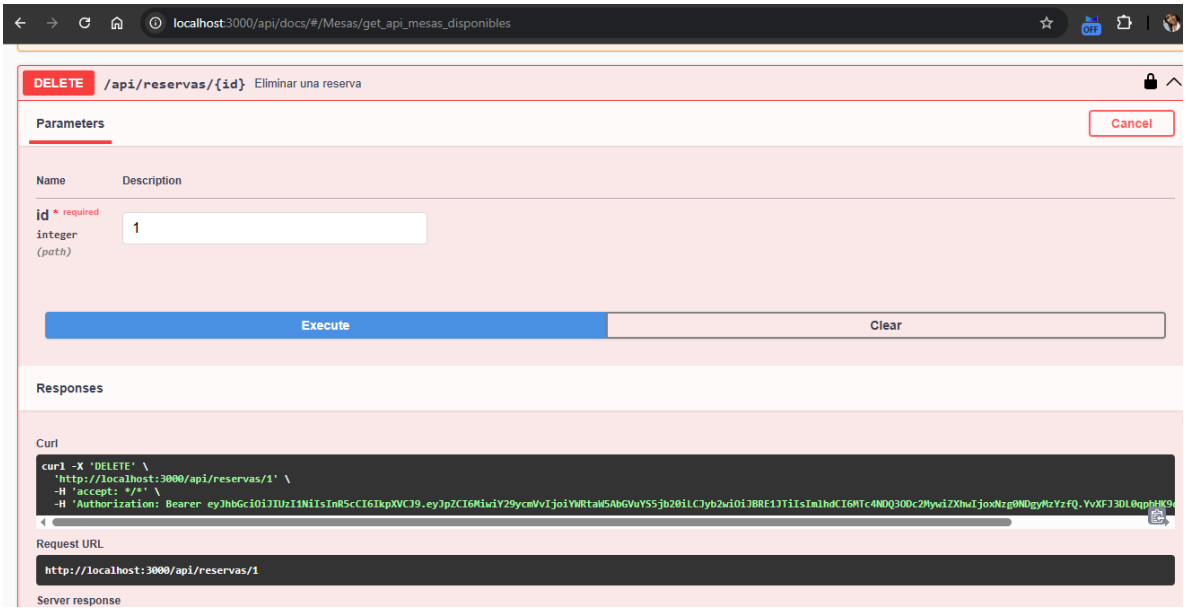
Eliminar reserva

Endpoint:

DELETE /api/reservas/1

Figura 20

Eliminación de una reserva



Nota: Eliminación de una reserva y liberación de la mesa relacionada (Vargas,2026).

La evidencia siguiente muestra la confirmación de la eliminación de una reserva, en la figura 21. Además, se asegura que la mesa asociada se libere automáticamente para futuras reservas.

Figura 21

Eliminación de reserva exitosa

Nota: Confirmación de que la reserva registrada ha sido eliminada correctamente (Vargas,2026).

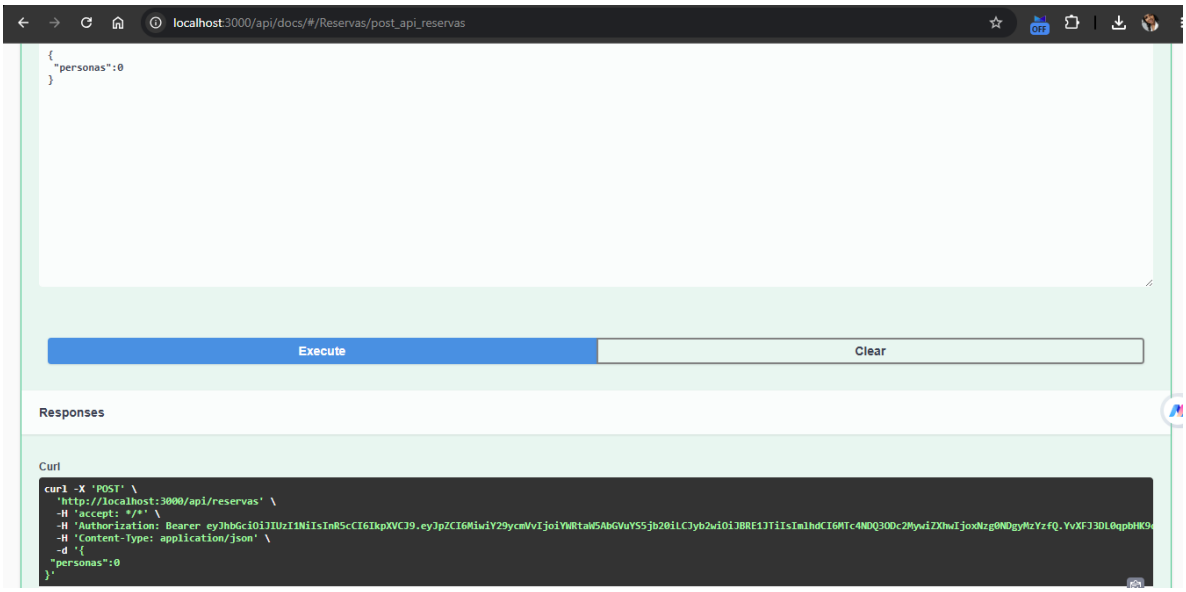
Códigos HTTP de validación

Error 400 - Datos incorrectos

POST reserva sin campos requeridos:

Figura 22

Respuesta con código HTTP 400

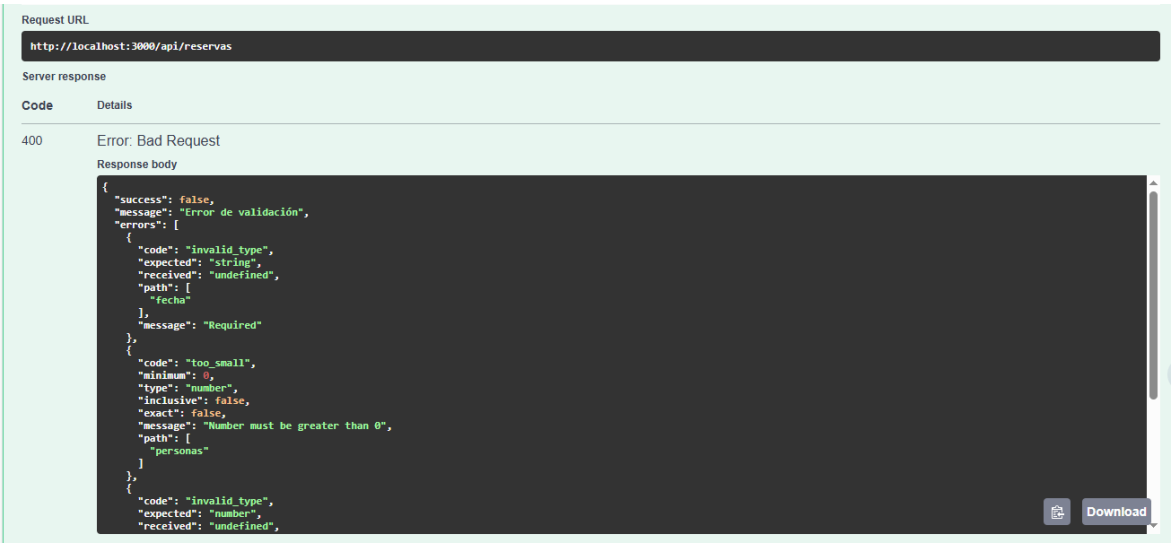


Nota: Ejemplo de un error generado por el envío de datos no válidos por parte del cliente (Vargas,2026).

La figura 23 que sigue presenta la validación de un error HTTP 400 generado por el envío de datos no válidos. Esta prueba verifica que las validaciones implementadas funcionan adecuadamente.

Figura 23

Validación del error HTTP 400



Nota: Respuesta emitida por el servidor a una solicitud que no cumple con las validaciones necesarias (Vargas,2026).

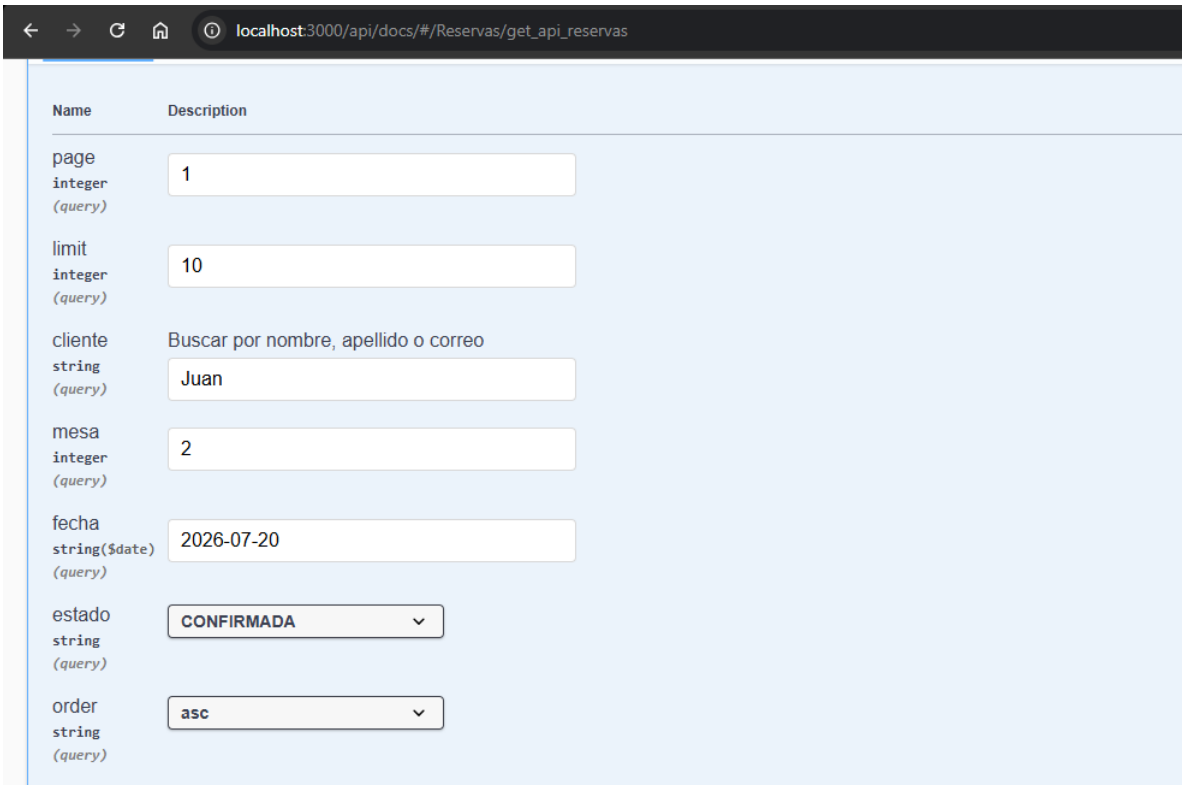
Error 401 - Sin autenticación

Solicitud sin token:

GET /api/reservas

Figura 24

Respuesta con código HTTP 401



Nota: Respuesta que ocurre cuando el usuario no presenta un token válido (Vargas,2026).

La evidencia a continuación corresponde a la validación del código HTTP 401 ante una solicitud que no incluyen un token de autenticación válido. Se demuestra que el mecanismo de seguridad usando JWT opera correctamente, en la figura 25.

Figura 25

Validación del error HTTP 401

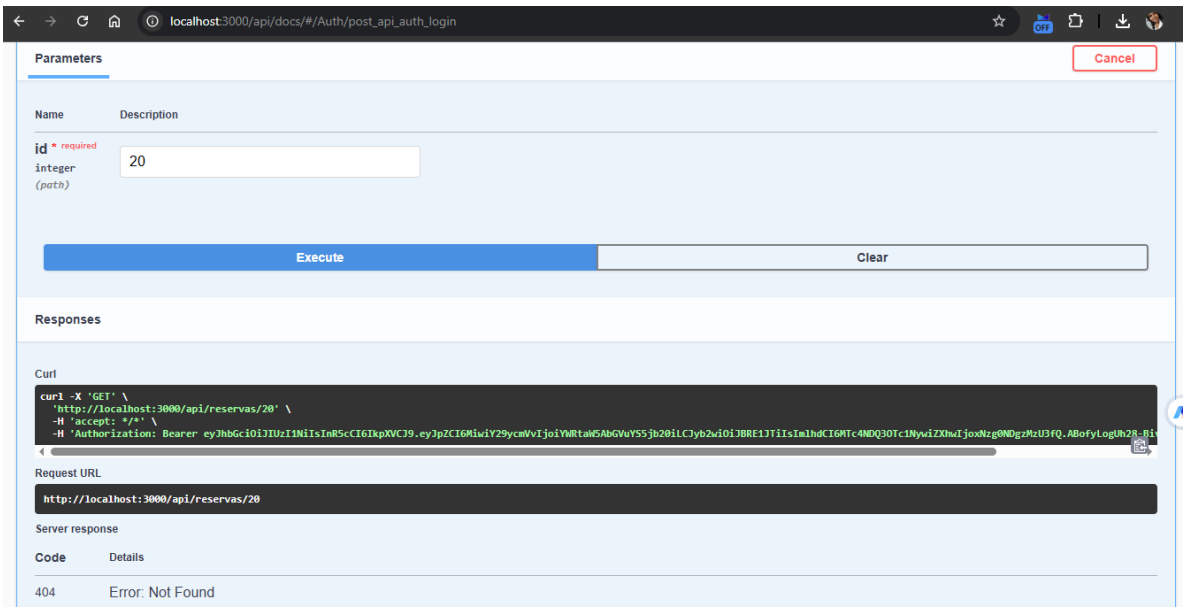
Nota: Comprobación de acceso denegado por falta de autorización adecuada (Vargas,2026).

Error 404 - Registro inexistente

La figura 26 que sigue presenta la respuesta del sistema a una solicitud de un recurso que no existe. Se confirma que se devuelve el código HTTP 404 como parte de la correcta gestión de errores.

Figura 26

Respuesta con código HTTP 404

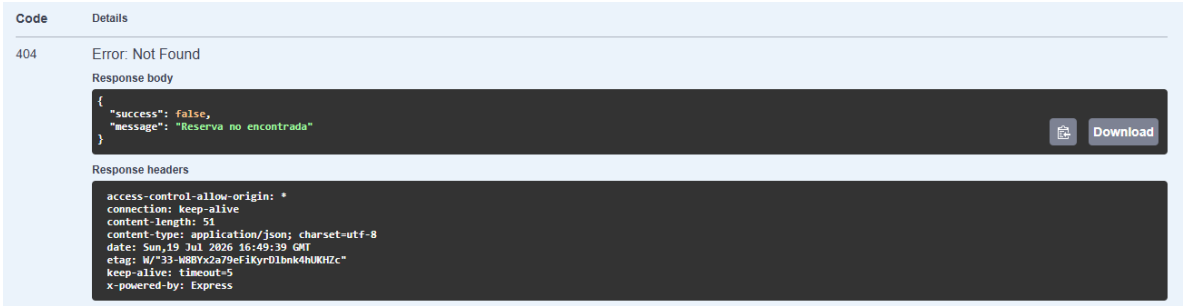


Nota: Error que se produce al intentar acceder a un recurso que no está presente en el sistema (Vargas,2026).

La evidencia que se presenta a continuación complementa la validación del código HTTP 404, confirmando que la API responde de manera adecuada cuando el recurso solicitado no se encuentra disponible, en la figura 27

Figura 27

Validación del error HTTP 404



Nota: Validación de la reacción del sistema frente a recursos que no existen (Vargas,2026).